CYPHER query language available in MultiGraphMatch

MATCH pattern [WHERE conditions] RETURN properties

MATCH clause

Proposition	Meaning
(a)	Find all nodes
()	Find all nodes
p = (a)	Find all nodes
(a:User)	Find all nodes of label 'User'
(:User)	Find all nodes of label 'User'
(a:User:Admin)	Find all nodes that are both 'User' and 'Admin'
(a:User|Admin)	Find all nodes that are either 'User' or 'Admin'
(a {name: 'Andres', sport:'Soccer'})	Find all nodes named 'Andres' and playing 'Soccer'
(a)-[]->(b)	Find all edges in a directed graph
()-[]->()	Find all edges in a directed graph
p = ()-[]->()	Find all edges in a directed graph
()-[r]->(b)	Find all edges in a directed graph
(a)<-[]->(b)	Find all bidirectional edges
(a)-[]-(b)	Find all edges (ignoring direction)
()-[]-(b)	Find all edges (ignoring direction)
(a)-[]->()-[]->(b)	Find all directed paths of length 2
p = (a)-[]->()-[]->(b)	Find all directed paths of length 2
(a:User)-[]->(b)	Find all directed edges outgoing from a node of label 'User'
(a)-[r:Friend {since:2001}]-(b)	Find all pairs of nodes that are friends since 2001
()-[r:Friend|Colleague]-()	Find all pairs of nodes that are either friends or colleagues
p = (a)-[]-(b)-[]-(a)	Find all couples of edges linking the same two nodes a and b
(a)-[]-(b); (a)-[]-(b); (a)-[]-(b)	Find all triplets of edges linking the same two nodes a and b

WHERE clause

Proposition	Meaning
id(a)=3	The ID of node or edge a is 3
id(a) IN [3,6,7]	The ID of node or edge a is 3, 6 or 7
a:User	a is a ‘User’
a:User:Admin	a is both a ‘User’ and an ‘Admin’
a.age<30	‘age’ of a is less than 30
a.age>=30	‘age’ of a is greater than or equal to 30
a.age<>30	‘age’ of a is not equal to 30
a.name=’Peter’	The name of a is ‘Peter’
a.name STARTS WITH ‘Pet’	The name of a starts with ‘Pet’
a.name ENDS WITH ‘er’	The name of a starts with ‘er’
a.name CONTAINS ‘ete’	The name of a contains substring ‘ete’
a.name IN [‘Andres’,’Peter’]	The name of a is in a list of names
a.name >= ‘Peter’	The name of a is ‘Peter’ or lexicographically follows Peter
exists(a.name)	Property ‘name’ exists for a
NOT exists(a.name)	Property ‘name’ does not exist for a
a.age IS NULL	‘age’ is an undefined property for a
a.age IS NOT NULL	‘age’ is a defined property for a
NOT a.name STARTS WITH ‘Pet’	The name of a does not starts with ‘Pet’
NOT a.age=30 AND a.name=’Peter’	a is not 30 years old and his name is ‘Peter’
NOT(a.age=30 AND a.name=’Peter’)	Neither a is 30 years old nor his name is ‘Peter’
a.name=’Peter’ OR b.age<30	Either the name of a is ‘Peter’ or b is less than 30 years old
a.name=’Peter’ AND b.age<30	The name of a is ‘Peter’ and b is less than 30 years old
a.name=’Peter’ AND (b.age<30 OR b.eyes=’green’)	The name of a is ‘Peter’ and either b is less than 30 years old or b’s eyes are green

RETURN clause

Proposition	Meaning
p	Return all nodes and all edges of occurrences of pattern p
a	Return all information about a (types and properties). If a is the name of a pattern, print all nodes and edges of the occurrences matching pattern a
a.age	Return the age of a
id(a)	Return the ID of node or edge a
labels(a)	Return all the class labels of node a
type(r)	Return all the types of an edge r
a.age, b.eyes	Return the age of a and the colour of b'eyes
nodes(p)	Return all the nodes of the occurrences matching pattern p
relationships(p)	Return all the edges of the occurrences matching pattern p
DISTINCT a	Return all information about a (types and properties) without repetitions. If a is the name of a pattern, select non-redundant occurrences matching pattern a and return all nodes and edges of such occurrences
count(*)	Count the number of pattern occurrences
count(a)	Count the number of occurrences of node, edge or pattern a (equivalent to count the number of occurrences)
count(a.name)	Count the number of occurrences of property "name" for node or edge a (equivalent to count the number of occurrences)
count(DISTINCT a.name)	Count the number of distinct values of property "name" for node or edge a
LIMIT 5	Stop searching when 5 occurrences have been found